

Relatório Técnico: Simulação Concorrente de Tráfego Urbano em C: Análise de Concorrência, Sincronização e Prevenção de Deadlocks

André Wesley Barbosa Rodrigues Filho¹, Leôncio Ferreira Flores Neto¹,
Paulo Gabriel Leite Landim¹, Salomão Rodrigues Silva¹

Universidade Federal do Cariri (UFCA)

Resumo

Este relatório apresenta o projeto e a implementação de um simulador de tráfego urbano concorrente em linguagem C utilizando a biblioteca de Threads (pthreads). Aborda-se a representação do mapa, o ciclo de vida multithreaded de veículos, e os mecanismos de sincronização empregados para garantir a exclusão mútua das células da via, com foco na eliminação da espera ocupada (*busy waiting*). Também discute a estratégia de *Look-Ahead* baseada em pré-reservas para prevenção de deadlocks e analisa a otimização da renderização da interface em console utilizando buffers acumulados (*double buffering*) com `snprintf`.

1. Introdução

Simular o tráfego urbano sob uma perspectiva de sistemas concorrentes exige mapear veículos físicos a fluxos independentes de execução (threads) que disputam recursos espaciais compartilhados [3] de capacidade restrita (as células da malha viária). Este trabalho descreve o projeto de um simulador concorrente implementado em C com a biblioteca nativa de threads [3], projetado para garantir a impenetrabilidade física dos veículos e a estabilidade da simulação livre de deadlocks (travamentos mútuos) e race conditions (condições de corrida) [2, 1].

2. Representação do Mapa

A inicialização da malha viária é feita de forma dinâmica através da leitura de um arquivo de texto volátil (`mapa.txt`). Essa flexibilidade arquitetural foi definida desde o início do projeto para que a equipe pudesse testar rapidamente diferentes layouts de ruas e cenários de estresse.

Uma vez lido, o mapa é traduzido logicamente para a memória como uma matriz bidimensional de estruturas `Cell`. A estrutura base foi implementada da seguinte forma:

```
1 typedef struct cell {
2     CellType type;           // empty, road, intersection
3     char direction;         // 'N', 'S', 'L', 'O' ou ' '
4     pthread_mutex_t mutex;   // lock de exclusao da celula
5     Vehicle *current_vehicle;
6 } Cell;
```

Cada célula individual possui atributos de tipo (estrada comum, cruzamento ou vazio), direção padrão de tráfego (Leste, Oeste, Norte, Sul) e um ponteiro opcional para a estrutura de

veículo que a ocupa no instante corrente. Esse ponteiro é essencial e utilizado exclusivamente para extrair os dados e desenhar o veículo corretamente na renderização visual da interface no console.

A escolha de associar um bloqueio (mutex) por célula, e não por mapa inteiro, ocorreu devido ao *trade-off* entre facilidade de implementação e melhor desempenho e paralelismo. É importante destacar também o impacto dessa decisão no custo de memória, que detalharemos melhor na subseção de Análise de Consumo (3.1).

Para estruturar a simulação, os elementos reais do trânsito foram mapeados para estruturas de código e recursos de concorrência, conforme ilustrado na Tabela 1.

Conceito Físico / Regra	Representação na Arquitetura C
Rua, Faixas e Mapa	Matriz bidimensional <code>Cell**</code> populada via leitura do <code>mapa.txt</code> .
Espaço Físico (1 Veículo por Célula)	Estrutura <code>Cell</code> com Mutex (<code>pthread_mutex_t</code>) para garantir a impenetrabilidade.
Veículos e Ambulância	Threads separadas (<code>pthread_create</code>) com ciclo de vida e planejamento independentes.
Tempo e Velocidade	Thread de Relógio autônoma disparando incrementos de <i>Ticks</i> (unidades discretas de tempo da simulação) globais.
Sinal Vermelho (Espera)	Variáveis de Condição (<code>pthread_cond_t</code>) para <i>adormecer</i> os carros sem gastar CPU (<i>busy waiting</i>).
Cruzamentos Críticos	Semáforos Contadores (<code>sem_t</code>) limitando a capacidade física máxima da zona de interseção.
Prioridade da Ambulância	Lógica de <i>Bypass</i> nos cruzamentos e <code>pthread_cond_broadcast</code> antecipado nos sinais.

Tabela 1: Mapeamento de Abstrações Físicas do Tráfego para Primitivas e Estruturas de Programação Concorrente

2.1. Trade-off de Granularidade do Lock

A modelagem adotada implementou exclusão mútua em nível de célula, onde cada estrutura `Cell` possui seu próprio Mutex (`pthread_mutex_t`).

- **Locks Grossos (Por rua ou mapa):** Reduziria a complexidade lógica do controle concorrente, mas impediria que carros distantes ocupassem a mesma avenida simultaneamente, destruindo a vazão de paralelismo.
- **Locks Finos (Por célula - Adotado):** Permite maior paralelismo de tráfego. Múltiplos veículos se movem concorrentemente nas mesmas faixas. Apresenta uma implementação simples e direta através do mapeamento um para um dos mutexes no grid de células. Contudo, introduz a necessidade de mecanismos rígidos para evitar colisões traseiras e gridlocks. Essa abordagem proporcionou maior paralelismo entre os veículos sem comprometer a segurança no acesso aos recursos compartilhados.
- **Locks Híbridos (Apenas Zonas Críticas):** Uma estratégia alternativa levantada foi a de mesclar e alocar mutexes apenas nas partes mais importantes e conflituosas do mapa (como

os cruzamentos), poupando espaço de memória ao longo das retas. Contudo, além de ser consideravelmente mais difícil de implementar do que o modelo de trava geral, dificultaria o controle lógico de colisões em vias contínuas.

3. Modelagem e Execução das Threads

O simulador executa de forma concorrente em três esferas principais:

- **Threads (fluxos de execução) de Veículos:** Cada carro ou ambulância no mapa é representado por uma thread individual cujo ciclo de vida consiste em ler sua posição atual, planejar o próximo movimento com base na direção, e disputar a posse da próxima célula.
- **Thread de Relógio (Clock):** Uma thread centralizadora que define o passo da simulação por meio de ticks regulares de tempo.
- **Thread do Gerenciador de Tráfego (spawn, geração dinâmica de veículos):** Coordena deterministicamente os ciclos dos semáforos em sintonia com o relógio global.

A velocidade discreta é respeitada pelas threads de veículos por meio da divisão de velocidade por ticks (veículos rápidos avançam a cada 1 tick, médios a cada 2 ticks e lentos a cada 4 ticks) conforme os conceitos clássicos de concorrência de CPU.

3.1. Análise de Consumo de Memória e o Peso dos Mutexes

Em teoria, a criação de uma nova thread solicita ao sistema operacional 8 MB de espaço de pilha virtual reservado. Em um cenário com 20 threads simultâneas ativas (como ocorre na execução do simulador com sua frota padrão), a reserva teórica de memória virtual apenas para as pilhas seria de 160 MB.

Na prática, medições empíricas com o simulador ativo — realizadas no Linux monitorando o processo em execução com o comando `ps` — registraram um consumo de memória virtual de aproximadamente 289,5 MB (que engloba as pilhas virtuais de todas as threads, o executável e as bibliotecas carregadas) e um consumo de memória física de apenas 2,02 MB. Esse valor de memória física comprova o mecanismo de alocação sob demanda (*lazy allocation*) de páginas pelo núcleo do Linux: como o ciclo de vida dos veículos é direto e não acumula estruturas volumosas, o gasto efetivo de memória física por thread fica em torno de poucos kilobytes, tornando a estratégia de utilizar um mutex por célula plenamente viável.

O peso das travas (Mutex): Como adotamos a estratégia de *Lock Fino* (um mutex por célula), há um custo. Cada trava ocupa 40 bytes. Se tivermos um mapa grande de 100x100 (10.000 células), gastaremos cerca de 400 KB de memória apenas com as travas. Se usássemos a estratégia do **Lock Híbrido** (sem travas nas retas), o gasto de memória da matriz do mapa diminuiria em cerca de **80% a 90%**. Contudo, como o programa é bastante leve (usa menos de 3 MB no total), a equipe decidiu que a facilidade e a segurança de ter travas em todas as células compensam os poucos 400 KB de custo extra.

O peso das travas (Mutex): Como adotamos a estratégia de *Lock Fino* (um mutex por célula), há um custo. Cada trava ocupa 40 bytes. Se tivermos um mapa grande de 100x100 (10.000 células), gastaremos cerca de 400 KB de memória apenas com as travas. Se usássemos a estratégia do **Lock Híbrido** (sem travas nas retas), o gasto de memória da matriz do mapa diminuiria em cerca de **80% a 90%**. Contudo, como o programa é bastante leve (usa menos de 3 MB no total),

a equipe decidiu que a facilidade e a segurança de ter travas em todas as células compensam os parcos 400 *KB* de custo extra.

3.2. Gerenciamento Dinâmico de Spawn (Geração Dinâmica de Veículos)

A estratégia de inserção de veículos na malha viária evoluiu ao longo do desenvolvimento do simulador, migrando de um modelo estático para um modelo dinâmico com coleta de lixo (*garbage collection*) de threads.

Abordagem Inicial (Spawn Estático): No início do projeto, todos os veículos eram criados de forma sequencial no instante de inicialização da simulação. Como trade-off, essa estratégia é computacionalmente simples e possui um consumo de recursos previsível e determinístico. Contudo, ela introduz um comportamento inadequado na simulação: como os veículos possuem velocidades distintas e rotas independentes, à medida que chegavam ao destino, suas threads finalizavam e eram destruídas. Após um curto período de execução, a simulação se tornava monótona e vazia com o fim dos veículos, restando apenas o grid inativo.

Abordagem Atual (Spawn Dinâmico): Para garantir a continuidade da simulação, implementou-se uma thread dedicada de gerenciamento de spawn (`spawn_thread`). Ela monitora constantemente a quantidade de veículos ativos e realiza a limpeza de threads finalizadas utilizando a chamada não-bloqueante `pthread_tryjoin_np`. Sempre que o número de carros ativos fica abaixo de um limite parametrizado (`limit_vehicles`), o gerenciador realiza tentativas de spawnar novos veículos nos pontos de entrada disponíveis no mapa, selecionados de maneira pseudoaleatória. Como trade-off, embora essa abordagem aumente a complexidade de sincronização (exigindo exclusão mútua na pool de veículos `fleet` e tratamento de desalocação de memória em tempo real), o modelo dinâmico viabiliza uma simulação contínua, realista e de execução indefinida, aproximando o simulador de um tráfego urbano real e mantendo a simulação ativa e dinâmica.

4. Mecanismos de Sincronização e Ausência de Espera Ocupada

A coordenação das threads concorrentes é realizada por meio de travas (`pthread_mutex_t`) e semáforos contadores (`sem_t`) [3, 2], evitando qualquer forma de desperdício de ciclos de CPU (espera ocupada / *busy waiting*) [1], conforme recomendado pela literatura de sistemas operacionais.

4.1. Exclusão Mútua, Capacidade de Cruzamento e Espera Bloqueante

O acesso exclusivo a cada célula física do mapa é garantido pelo mutex interno da célula. Para gerenciar a capacidade máxima de veículos dentro das zonas críticas de cruzamento e evitar superlotação, utilizou-se semáforos clássicos (`sem_t`).

A estrutura de sincronização de capacidade funciona nos seguintes moldes:

```

1 // Tenta reservar vaga no cruzamento (nao-bloqueante)
2 bool traffic_try_enter_capacity(int row, int col) {
3     TrafficLight *tl = get_light_at(row, col);
4     if (!tl) return true; // Celulas comuns sem limite
5
6     TrafficLight *base = tl->base_light ? tl->base_light : tl;
7     return (sem_trywait(&base->capacity_sem) == 0);
8 }
9
10 // Libera vaga no cruzamento ao sair
11 void traffic_release_capacity(int row, int col) {
12     TrafficLight *tl = get_light_at(row, col);
13     if (!tl) return;
14
15     TrafficLight *base = tl->base_light ? tl->base_light : tl;
16     sem_post(&base->capacity_sem);
17 }

```

A expressão ternária `tl->base_light ? tl->base_light : tl` é utilizada para desviar o controle de fluxo para a célula superior-esquerda (célula base) do cruzamento 2x2 caso ela tenha sido identificada e atribuída; caso contrário, a própria célula atual assume o papel de base (como ocorre em cruzamentos isolados de dimensão 1x1).

Além disso, para evitar o *busy waiting* (espera ocupada) de veículos esperando nos sinais vermelhos, o simulador utiliza variáveis de condição (`pthread_cond_t`) [3, 2]. Ao detectar o sinal vermelho, a thread do veículo chama `pthread_cond_wait`, liberando temporariamente a CPU e entrando em estado de repouso (*sleeping*). Quando o gerenciador de tráfego altera o sinal para verde, ele executa um `pthread_cond_broadcast`, acordando passivamente todas as threads bloqueadas.

4.2. Trade-off de Transição de Semáforos

A lógica de transição dos semáforos foi projetada para ser imune a race conditions:

- **Threads Ativas Desacopladas:** Se cada semáforo operasse sua própria temporização e acordasse carros concorrentemente, o escalonamento do sistema operacional geraria inconsistências visuais e de lógica de tráfego sob carga pesada.
- **Cálculo Determinístico pelo Tick (Adotado):** O estado do semáforo é calculado de forma puramente matemática a partir do tick global de tempo. Quando a fórmula indica a necessidade de transição, o semáforo aciona a função `pthread_cond_broadcast`. Isso garante que nenhum carro tome uma decisão de travessia baseada em estados intermediários ou defasados.

5. Estratégia Contra Deadlocks e Ambulância

O maior desafio concorrente da simulação reside nos cruzamentos 2x2, onde a necessidade de posse cumulativa de múltiplos recursos espaciais (as células da interseção e a célula de saída) cria as condições de espera circular de Coffman (deadlock).

5.1. Estratégia Look-Ahead Preventiva

Para mitigar deadlocks, implementou-se o algoritmo de Look-Ahead preventivo. Antes de entrar em qualquer área de cruzamento, a thread de veículo precisa pré-reservar não apenas a

célula da interseção, mas também a célula de saída após o cruzamento utilizando chamadas não-bloqueantes `pthread_mutex_trylock`. Caso a saída esteja congestionada ou inacessível, o veículo não entra no cruzamento, evitando o travamento da interseção. Antes de entrar em um cruzamento, o veículo verifica se será possível concluir toda a travessia. Caso a célula de saída esteja ocupada, ele permanece aguardando antes da entrada, evitando bloquear o cruzamento.

5.2. Bypass de Interseção e Ambulância

Durante a simulação concorrente com veículos de prioridade (Ambulância), identificou-se o seguinte trade-off:

- **Look-Ahead Estrito Interno:** Carros civis já inseridos nas células do cruzamento paravam nos semáforos internos, trancando a passagem. Como a ambulância se aproximava, formava-se um deadlock eterno.
- **Bypass Interno (Adotado):** Carros que já se encontram dentro da área física de interseção ignoram as checagens normais de semáforos vermelhos ao avançar para a célula de saída. Essa regra de bypass limpa os cruzamentos com prioridade ao tráfego da ambulância.

Para a ambulância, foi implementada a prioridade preventiva em bloco: ao se aproximar, ela solicita passagem e força o sinal verde para todo o bloco 2x2 adjacente da interseção, liberando o cruzamento em bloco após a sua saída completa da área.

6. Desempenho da Interface e Impacto na Simulação

A visualização do trânsito na tela do console exige a impressão em formato ASCII da malha inteira em cada tick de tempo discreto.

6.1. O Gargalo de I/O de console em tempo real

No design inicial, o display iterava sobre a matriz do mapa realizando chamadas síncronas de `printf` para cada caractere impresso no terminal. Isso gerava atrasos de até 150 ms por renderização. Como a thread de renderização disputava mutexes do mapa com as threads de veículos, os carros sofriam atrasos artificiais na sua movimentação (latência propagada).

6.2. Double Buffering via `snprintf`

A solução foi a implementação de um buffer de quadro (Double Buffering) em memória RAM.

```
1 char buffer[32768];
2 int offset = 0;
3 // Acumula os caracteres do mapa na memoria RAM
4 offset += snprintf(buffer + offset, size, "%c", ch);
5 // Realiza apenas um print final por tick
6 printf("%s", buffer);
```

Essa mudança reduziu a latência do I/O síncrono no terminal, reduzindo o tempo de renderização do display para menos de 1ms e permitindo que as threads de veículos executassem em tempo de CPU real de forma contínua e estável.

6.3. Visualização dos Comportamentos Observados

As capturas a seguir ilustram os principais comportamentos concorrentes da simulação em execução real no terminal. A interface é atualizada a cada tick via *double buffering*, exibindo a posição dos veículos, o estado dos semáforos e os alertas de prioridade. As Figuras 1, 2 e 3 demonstram, respectivamente, o estado geral da malha viária, a prioridade da ambulância e o bloqueio de veículos em sinal vermelho.

```
=== SIMULADOR DE TRAFEGO PTHREAD === (Tick: 23)
Legenda: [C] Carro | [A] Ambulancia | [-] Verde Horizontal | [I] Verde Vertical

      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
      LLLLLL--LLLCLLL|LLALLL|LLLLLLL|LLLLLLL
      000000--000000|000000C|000000|000000
      NS      CC      NC      NS
      NS      NS      NS      NS
      CS      NS      NS      NS
      LLLLLL|LLLLLL--LLLLLL--LLLLLL|LLLLLL
      000000|000000--000000C--000000|000000
      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      CS      NS
      LLLLLL|LLLLLL|LCLLLL|LLLLLL--LLLLLL
      000000|0C0000|000000|000000--000000
      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
```

Figura 1: Estado geral do simulador (Tick 23). Carros civis (C) circulam pelas vias horizontais (L/O) e verticais (N/S). Cruzamentos com -- indicam sinal verde para o eixo horizontal; com || indicam verde vertical.

```
=== SIMULADOR DE TRAFEGO PTHREAD === (Tick: 9)
Legenda: [C] Carro | [A] Ambulancia | [-] Verde Horizontal | [I] Verde Vertical
[ALERTA] Prioridade MAXIMA! Ambulancia forçando a passagem num cruzamento!

      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NC      NC      NS
      LLLLLLA-CLLLLL--LLLLLL--LLLLLL|LLLLLL
      000000--000000--000000--000000C|000000
      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
      LLLLLL|LLLLLL|LLLLLL|LLLLLL--LLLLLL
      000000|C000000|000000|000000C-000000
      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
      LLLLLLCC-LLLLLL--LLLLLL|LLLLLL|LLLLLL
      000000--000000--000000|000000|00C0000
      NS      CS      CS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
```

Figura 2: Ambulância com prioridade (Tick 9). Alerta ativo via `traffic_request_priority`. Cruzamentos horizontais forçados para verde (--); via vertical bloqueada.

```

=== SIMULADOR DE TRAFEGO PTHREAD === (Tick: 7)
Legenda: [C] Carro | [A] Ambulancia | [-] Verde Horizontal | [|] Verde Vertical

      NS      NS      NS      NS
      NS      NC      NC      NS
      NS      NS      NS      NS
      LLLLLL|C|LLLLLL|LLLLLL--LLLLLL--LLLLLL
      0000000|0000000|0000000--0000000--C000000
      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
      LLLLLL--LLLLLL|LLLLLL|LLLLLL|LLLLLL
      0000000--0000000|0000000|0000000|C000000
      NS      NS      NS      NS
      NS      NS      NS      NS
      NS      NS      NS      NS
      LLLLLL|C|LLLLLL--LLLLLL--LLLLLL|LLLLLL
      0000000C|0000000--0000000--0000000|000C000
      NS      CS      NS      NS
      NS      NS      CS      NS
      NS      NS      NS      NS

```

Figura 3: Respeito ao sinal vermelho (Tick 7). Carros horizontais (C) parados antes de cruzamentos com sinal || (verde vertical) bloqueados via `pthread_cond_wait`.

7. Conclusão e Divisão de Responsabilidades

O simulador concorrente desenvolvido atingiu plenamente os objetivos propostos ao modelar o fluxo de tráfego urbano utilizando a biblioteca Pthreads. A aplicação integrada de mutexes, semáforos e variáveis de condição garantiu exclusão mútua por célula, controle de capacidade em cruzamentos e repouso bloqueante sem desperdício de CPU (*busy waiting*). Além disso, a estratégia preventiva *Look-Ahead* mostrou-se eficaz na eliminação de travamentos mútuos (*deadlocks*) e a otimização por *double buffering* solucionou os gargalos de renderização do console. A integridade da simulação foi assegurada por testes unitários e pelo pipeline de integração contínua (CI).

A implementação do simulador concorrente foi dividida de maneira organizada, baseando-se no cronograma das tarefas registradas, com as seguintes atribuições:

Integrante	Responsabilidade Principal
Paulo Gabriel	Parser de mapa, inicialização da malha viária, lógica de semáforos, sincronização de tráfego e resolução de bugs de concorrência na interface.
Salomão Rodrigues	Módulo de relógio (sincronização temporal), motor de visualização ASCII (display) e gerenciador dinâmico de spawn (geração dinâmica de veículos).
André Wesley	Orquestração central (<code>main.c</code>), liberação de recursos, tratamento de sinais (Ctrl+C), estratégia anti-deadlock e módulo de logging concorrente.
Leôncio Ferreira	Estruturação arquitetural, threads do ciclo de vida dos veículos (civis e ambulância), refatorações de simulação e escrita do artigo.

Tabela 2: Mapeamento Final de Execução das Tarefas da Equipe

Referências

- [1] William P. Alves. *Sistemas operacionais - 1a edição*. SRV Editora LTDA, São José dos Campos, SP, 2014. E-book. ISBN 9788536531335.
- [2] Francis B. Machado and Luiz P. Maia. *Fundamentos de Sistemas Operacionais*. Grupo GEN, Barueri, SP, 2011. E-book. ISBN 978-85-216-2081-5.
- [3] Andrew S. Tanenbaum and Albert S. Woodhull. *Sistemas operacionais*. Grupo A, Porto Alegre, RS, 2008. E-book. ISBN 9788577802852.